Early Evidence on Generative AI Exposure and Labor Market Outcomes

Team Members: Aesha Gandhi, Pinaki Ghosh, Shambhavi Khanna, Isaac Vergara
Stakeholder: Director of Workforce Policy, State Department of Labor
Repository: `UDS-Project`

Stakeholder Summary

This memo is written for a Director of Workforce Policy at a state Department of Labor. The intended stakeholder is assumed to be comfortable interpreting applied research and labor market indicators, but not interested in econometric detail for its own sake. The main priority is actionable evidence that can inform workforce planning and early-career employment policy.

Executive Summary / TL;DR

This memo examines whether workers in industries with higher exposure to generative AI experienced different labor market outcomes after the arrival of GenAI tools, and whether those changes were concentrated among younger workers. The analysis uses CPS ASEC data from 2016-2025 merged with an industry-level AI exposure measure, `AIIE`, and estimates difference-in-differences models for unemployment and wages.

The main result is that after 2023, workers in higher-AI-exposure industries experienced a small but statistically significant relative increase in unemployment. This does not mean those industries suddenly became high-unemployment industries. In fact, they still tend to have lower unemployment overall. Rather, their pre-existing unemployment advantage narrowed in the post-GenAI period. When the model allows the effect to vary by age, the increase is concentrated among early-career workers, especially the youngest part of that group. This pattern is more consistent with weaker entry-level hiring than with broad displacement of experienced workers.

No comparable wage effect is detected. The wage results are statistically insignificant and should be treated more cautiously because the pre-trend diagnostics are weaker in that specification. Overall, the most credible conclusion is that generative AI exposure is associated with modestly worse unemployment outcomes for younger workers in exposed industries, while wage effects are not yet detectable over this short post period.

Decisions To Be Made

Based on these findings, three near-term decisions appear especially relevant:

1. Whether to prioritize early-career employment programs in AI-exposed sectors, especially for workers around ages 25-29.
2. Whether to monitor hiring and job-finding rates, not just layoffs, when evaluating the labor market impact of AI.
3. Whether to treat wages as a lagging indicator for now and focus policy attention more on access to entry-level opportunities.

The Problem

Much of the public discussion around generative AI focuses on whether AI will eliminate jobs or lower wages. For a labor market policymaker, however, the more immediate question is narrower and more practical: are workers in more AI-exposed industries already showing signs of weaker labor market outcomes, and if so, which workers are most affected?

This matters because if the main impact of AI operates through reduced hiring into exposed occupations, then the workers most at risk may not be incumbent workers with long tenure. Instead, the burden may fall on newer workers trying to enter the labor market. That would imply a different policy response than one focused mainly on retraining laid-off workers.

Analytical Approach

To study this question, CPS ASEC data from 2016-2025 were merged with an industry-level AI exposure measure, `AIIE`. Repeated-cross-section difference-in-differences models were then estimated.

The basic idea is straightforward. Industries differ in how exposed they are to AI, but that exposure is mostly fixed over time. The relevant question is therefore whether outcomes changed differently after the GenAI shock in industries with higher AI exposure relative to industries with lower AI exposure.

The analysis focuses on two outcomes:

- Unemployment
- Wage income

A version of the unemployment model was also estimated that allows the post-period AI exposure effect to differ for early-career workers.

Greater weight is placed on the unemployment results because the parallel trends evidence is more supportive in that specification. The wage results are still reported, but they should be interpreted as weaker and more tentative.

Descriptive Patterns

The descriptive trends help establish the baseline pattern before turning to the causal models. High-AI-exposure industries generally have lower unemployment than low-exposure industries throughout the sample, but that gap becomes narrower after the GenAI period begins. By contrast, wage trends are flatter and do not show an obvious post-period break.

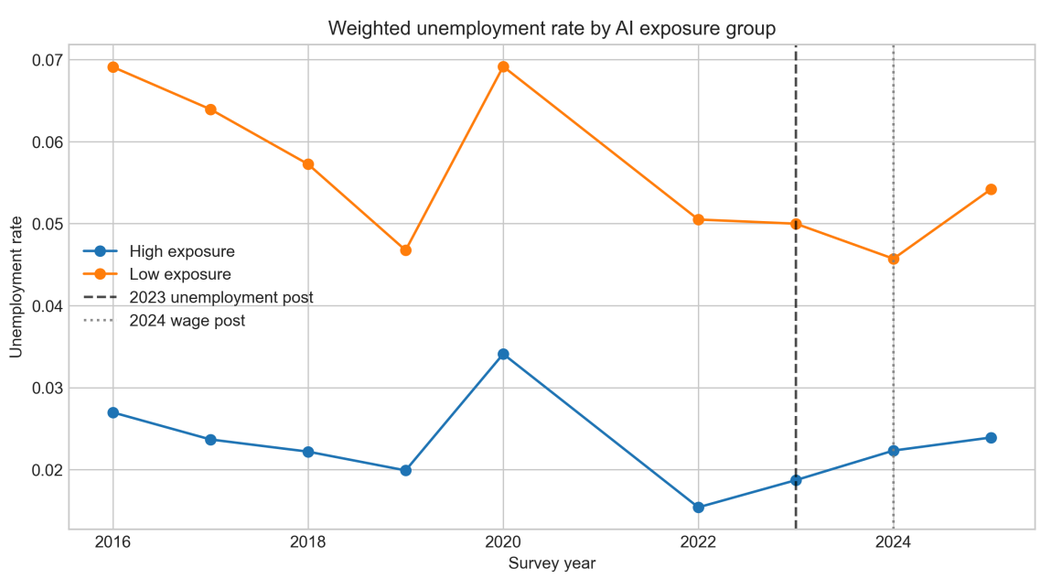


Figure 1. Weighted unemployment rates by AI exposure group. The post-2023 narrowing of the unemployment gap motivates the difference-in-differences analysis.

The age-focused descriptive plots also suggest that the relative unemployment shift is more pronounced for younger workers than for older workers, which is consistent with a hiring-channel interpretation.

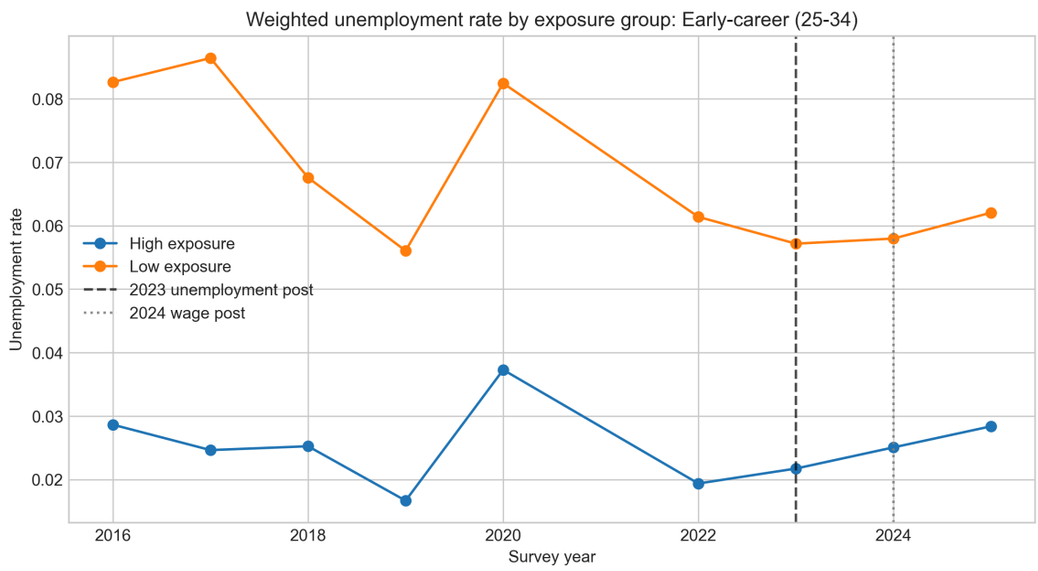


Figure 2. Weighted unemployment rates for early-career workers ages 25-34.

Figure 3. Weighted unemployment rates for older workers ages 50-64. The shift is visibly smaller than in the early-career group.

The wage plot is included for completeness, but it provides much less visual evidence of a clear post-period change by exposure group.

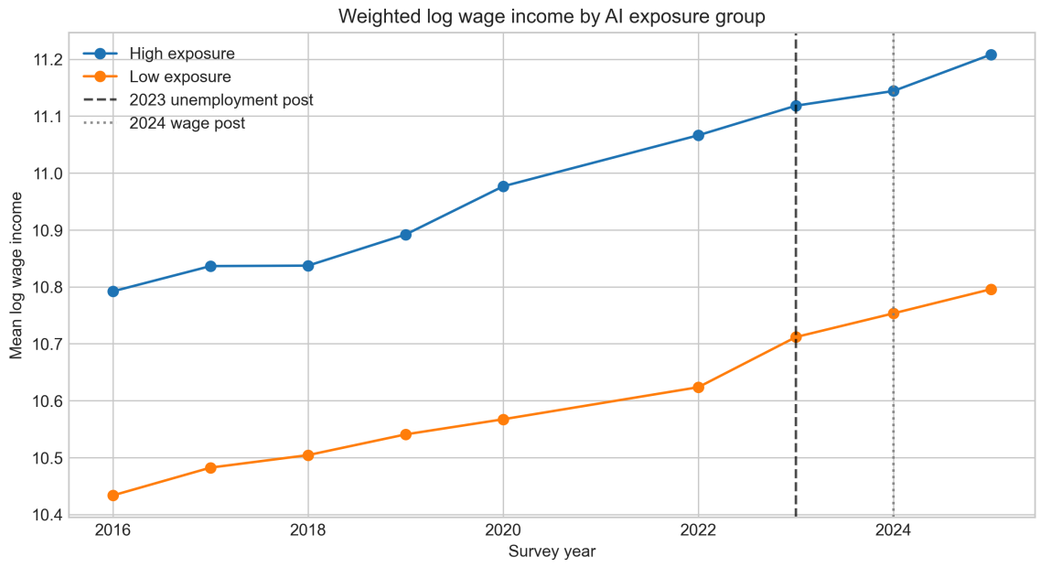


Figure 4. Weighted log wage income by AI exposure group. Clear post-period divergence is not apparent.

Main Findings

1. Higher AI exposure is associated with a relative increase in unemployment after 2023

The baseline unemployment DiD coefficient on `AIIE × post` is positive and statistically significant, around `0.003` (`p = 0.007`). This implies that after 2023, unemployment rose slightly more, or fell less, in higher-AI-exposure industries than in lower-exposure industries.

This is a modest effect, not a dramatic one. It also does not mean exposed industries became high-unemployment industries overall. In the raw data, those industries still tend to have lower unemployment than less-exposed industries. The key point is that their advantage narrowed in the post period.

2. The unemployment effect is concentrated among early-career workers

In the triple-interaction model, the coefficient on `AIIE × post × early_career` is positive and statistically significant. This suggests the post-period AI exposure effect is stronger for workers ages 25-34 than for older workers.

This result is important because it points to where the labor market adjustment is happening. If the effect were evenly spread across all workers, broad displacement would appear more plausible. Because it is stronger for younger workers, a more plausible interpretation is a hiring channel: firms in more AI-exposed industries may be slowing entry-level hiring rather than laying off experienced workers at large scale.

3. The youngest subgroup appears to drive the pattern

When the early-career group is split further, the strongest signal comes from workers ages 25-29. The 30-34 group appears weaker and is not clearly significant.

This makes the hiring-channel interpretation more credible. Workers closer to labor market entry seem to be absorbing more of the shock than workers who are slightly older and more established.

4. No meaningful wage effect is detected

In the wage DiD, the main interaction term `AIIE × post_wage` is small and statistically insignificant. There is therefore no evidence that wages changed differently after 2024 in higher-AI-exposure industries.

This should not be over-interpreted as proof that AI has no wage effects. The post period is still short, and wages may adjust more slowly than employment. In addition, the wage specification rests on weaker identifying assumptions because the pre-trend evidence is less reassuring.

Why the Unemployment Result Is More Credible Than the Wage Result

The parallel trends assumption was evaluated using event-study style models. For unemployment, the pre-period coefficients were mostly small and statistically insignificant, although not perfectly flat. This is not a perfect pass, but it is supportive enough for the unemployment DiD to be interpreted as reasonably credible.

For wages, the pre-period coefficients were consistently negative and statistically significant for several years before the post period. That means wage outcomes were already evolving differently across high- and low-exposure industries before GenAI. Because of that, the wage model does not support as strong a causal interpretation.

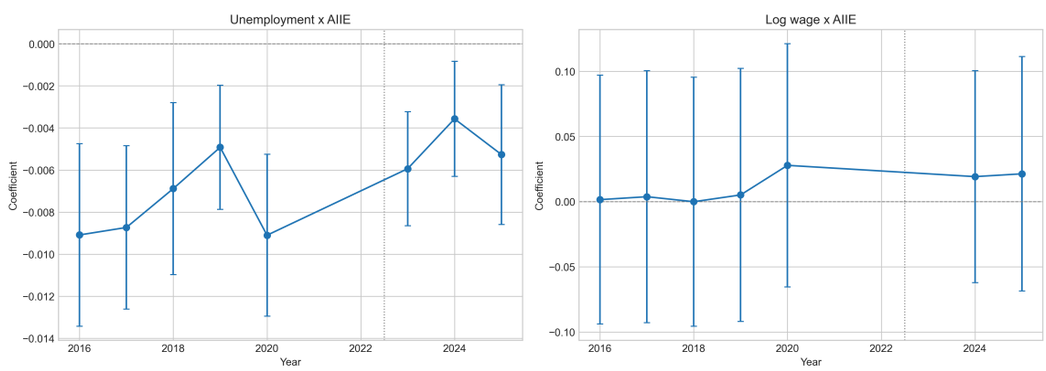


Figure 5. Event-study comparison. The unemployment pre-period is imperfect but much closer to parallel trends than the wage pre-period.

The robustness check that drops 2020 reaches a similar conclusion. The unemployment pattern remains visible, which reduces concern that the result is being driven entirely by pandemic-era volatility.

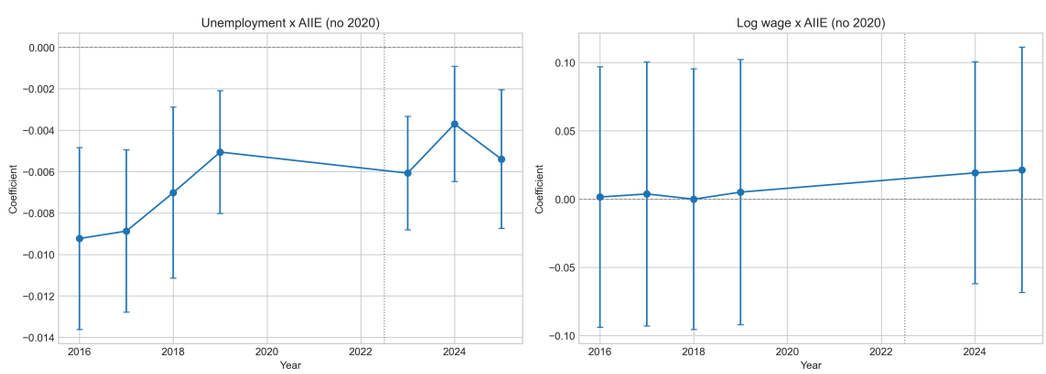


Figure 6. Event-study comparison excluding 2020. The unemployment pattern remains more credible than the wage pattern.

Robustness

The unemployment result was also tested by dropping 2020, the most unusual COVID year. The main unemployment pattern remains, and if anything the point estimates become slightly larger. This reduces concern that the result is only a pandemic artifact.

The event-study patterns were also examined visually. For unemployment, the pre-period is not perfectly flat, but it looks much more reasonable than the wage case, and the main post-period shift appears after the GenAI timing break.

Limitations

This analysis has meaningful limitations that matter for interpretation.

First, the CPS is a repeated cross-section, not a panel, so the same worker cannot be tracked over time. The analysis therefore captures changes in group-level outcomes rather than individual trajectories.

Second, `AIIE` is a measure of potential AI exposure based on industry task content, not actual AI adoption. The analysis therefore does not observe whether a specific employer actually implemented GenAI tools.

Third, the post period is still short. For wages in particular, the data may simply not yet have had enough time to reflect meaningful adjustment.

Finally, the wage specification does not satisfy the parallel trends assumption as cleanly as the unemployment specification, so the wage null result should not be treated as final evidence.

Conclusion

The central conclusion is that generative AI exposure is associated with a modest relative increase in unemployment after 2023, and that this effect is concentrated among early-career workers, especially those ages 25-29. Comparable wage effects are not yet detectable.

For this stakeholder, the more useful framing is not a broad "AI destroys jobs" narrative, but whether AI is weakening entry points into exposed industries. That suggests a practical policy agenda centered on early-career pathways, job-finding, internships, and first-job transitions rather than relying only on retraining incumbent workers after displacement.

Technical Appendix

This appendix is included to show the analytical work underlying the memo. It documents how the dataset was assembled and how the causal models were estimated.

The appendix focuses on the final analytical workflow used in this project:

- `data_merging.ipynb` for dataset construction and crosswalk logic
- `models/causal_analysis.ipynb` for sample creation, weighted descriptive analysis, parallel trends checks, DiD estimation, and robustness checks

Appendix A. Data Assembly Code

The code below shows how the CPS data were merged with industry-level AI exposure (`AIIE`) and occupation-level AI exposure (`AIOE`) using the Census-to-NAICS and Census-to-SOC crosswalks.

In [18]:
```
import re
import pandas as pd

aiie = pd.read_excel("../data/AIOE_DataAppendix.xlsx", sheet_name="Appendix
B")

crosswalk_clean = (
    pd.read_excel(
        "../data/2022-Census-Industry-Code-List-with-Crosswalk.xlsx",
        sheet_name="2022 Census Ind Code List ",
    )
    .dropna(how="all")
    .rename(
        columns={
            "Industry 2022 Description": "industry_desc",
            "2022 Census Industry Code": "census_ind_raw",
            "2022 NAICS Code": "naics_raw",
        }
    )
)

is_leaf = ~crosswalk_clean["census_ind_raw"].astype(str).str.contains("-",
na=False)
crosswalk_clean = crosswalk_clean.loc[is_leaf].copy()
crosswalk_clean["IND"] = pd.to_numeric(
    crosswalk_clean["census_ind_raw"], errors="coerce"
).astype("Int64")
crosswalk_clean = crosswalk_clean.dropna(subset=["IND"])


def parse_naics_patterns(raw):
    if pd.isna(raw):
        return []
    s = str(raw)
    s = re.sub(r"(?i)part of|pt\.?", "", s)
    s = re.sub(r"(?i)exc\..*", "", s)
    patterns = []
    for chunk in s.split(","):
        m = re.match(r"\s*(\d+)", chunk)
        if m:
            patterns.append(m.group(1))
    return patterns


crosswalk_clean["naics_patterns"] = crosswalk_clean["naics_raw"].apply(
    parse_naics_patterns
)
```

```
code_changes = (
    pd.read_excel(
        "../data/2022-Census-Industry-Code-List-with-Crosswalk.xlsx",
        sheet_name="2022 Ind Code Changes",
        header=2,
    )
    .dropna(subset=["2017 Census Industry Code"])
    .astype({"2017 Census Industry Code": int, "2022 Census Industry Code":
int})
)
ind_2017_to_2022 = dict(
    zip(
        code_changes["2017 Census Industry Code"],
        code_changes["2022 Census Industry Code"],
    )
)
```

In [19]:
```
aiie_lookup = dict(zip(aiie["NAICS"].astype(str), aiie["AIIE"]))
aiie_keys_longest_first = sorted(aiie_lookup, key=len, reverse=True)


def ind_to_aiie(patterns):
    narrow = [
        aiie_lookup[k]
        for p in patterns
        for k in aiie_keys_longest_first
        if k.startswith(p)
    ]
    if narrow:
        return sum(narrow) / len(narrow)
    for p in patterns:
        for k in aiie_keys_longest_first:
            if p.startswith(k):
                return aiie_lookup[k]
    return None


crosswalk_clean["AIIE"] =
crosswalk_clean["naics_patterns"].apply(ind_to_aiie)
ind_aiie = (
    crosswalk_clean.dropna(subset=["AIIE"])
    .astype({"IND": "int64"})[["IND", "industry_desc", "AIIE"]]
    .reset_index(drop=True)
)

df = pd.read_csv("../data/cps_00001.csv")
df["IND_2022"] = df["IND"].map(lambda x: ind_2017_to_2022.get(x, x))
df_aiie = df.merge(
    ind_aiie.rename(columns={"IND": "IND_2022"}),
    on="IND_2022",
    how="left",
)
```

In [20]:
```
aioe = pd.read_excel("../data/AIOE_DataAppendix.xlsx", sheet_name="Appendix
A")

occ_crosswalk = (
    pd.read_excel(
        "../data/2018-occupation-code-list-and-crosswalk.xlsx",
        sheet_name="2018 Census Occ Code List",
        header=0,
    )
    .dropna(how="all")
    .rename(
        columns={
            "2018 Census Title ": "occ_desc",
            "2018 Census Code": "census_occ_raw",
            "2018 SOC Code": "soc_raw",
        }
    )
)
```

```
is_leaf = ~occ_crosswalk["census_occ_raw"].astype(str).str.contains("-",
na=False)
occ_crosswalk = occ_crosswalk.loc[is_leaf].copy()
occ_crosswalk["OCC"] = pd.to_numeric(
    occ_crosswalk["census_occ_raw"], errors="coerce"
).astype("Int64")
occ_crosswalk = occ_crosswalk.dropna(subset=["OCC"])


def parse_soc_pattern(raw):
    if pd.isna(raw):
        return ""
    m = re.match(r"^([\d\-]+)", str(raw).strip())
    return m.group(1) if m else ""


occ_crosswalk["soc_pattern"] =
occ_crosswalk["soc_raw"].apply(parse_soc_pattern)

aioe_lookup = dict(zip(aioe["SOC Code"].astype(str), aioe["AIOE"]))
aioe_keys_longest_first = sorted(aioe_lookup, key=len, reverse=True)


def occ_to_aioe(pattern):
    if not pattern:
        return None
    narrow = [aioe_lookup[k] for k in aioe_keys_longest_first if
k.startswith(pattern)]
    if narrow:
        return sum(narrow) / len(narrow)
    if pattern.endswith("0"):
        broad = pattern[:-1]
        group = [aioe_lookup[k] for k in aioe_keys_longest_first if
k.startswith(broad)]
        if group:
            return sum(group) / len(group)
    for k in aioe_keys_longest_first:
        if pattern.startswith(k):
            return aioe_lookup[k]
    return None


occ_crosswalk["AIOE"] = occ_crosswalk["soc_pattern"].apply(occ_to_aioe)
occ_aioe = (
    occ_crosswalk.dropna(subset=["AIOE"])
    .astype({"OCC": "int64"})[["OCC", "occ_desc", "AIOE"]]
    .reset_index(drop=True)
)
```

In [21]:

```
occ_changes = (
    pd.read_excel(
        "../data/2018-occupation-code-list-and-crosswalk.xlsx",
        sheet_name="Occ Code Changes",
        header=2,
    )
    .rename(
        columns={
            "2010 Census OCC Code": "occ_2010",
            "2010 Census Title": "title_2010",
            "New 2018 Census OCC Code": "occ_2018",
        }
    )
    .dropna(how="all")
)

occ_changes[["occ_2010", "title_2010"]] = occ_changes[
    ["occ_2010", "title_2010"]
].ffill()
occ_changes = occ_changes.dropna(subset=["occ_2010", "occ_2018"]).astype(
    {"occ_2010": int, "occ_2018": int}
)
```

```
occ_2018_to_aioe = dict(zip(occ_aioe["OCC"], occ_aioe["AIOE"]))
occ_changes["AIOE"] = occ_changes["occ_2018"].map(occ_2018_to_aioe)

pre2018 = (
    occ_changes[~occ_changes["occ_2010"].isin(occ_aioe["OCC"])]
    .dropna(subset=["AIOE"])
    .groupby("occ_2010", as_index=False)
    .agg(occ_desc=("title_2010", "first"), AIOE=("AIOE", "mean"))
    .rename(columns={"occ_2010": "OCC"})
)

occ_aioe_full = pd.concat([occ_aioe, pre2018], ignore_index=True)
df_aiie = df_aiie.merge(occ_aioe_full, on="OCC", how="left")

df_aiie.to_csv("../data/data_with_aiie.csv", index=False)
```

Appendix B. Analysis Setup and Variable Construction

The next blocks reproduce the final causal analysis setup, including reproducibility settings, sample restrictions, variable definitions, and helper functions used throughout the analysis.

Analysis Frame and Key Variables

The final analysis uses CPS ASEC repeated cross-sections from 2016-2025 merged with industry-level AI exposure, `AIIE`. The analytical frame applies the following restrictions and variable definitions:

- Working-age sample: ages 25-64
- Exposure requirement: non-missing `AIIE`
- Unemployment sample: individuals in the labor force
- Wage sample: employed individuals with positive wage income
- Main wage DiD sample: same as the wage sample, excluding 2023 as a transition year
- Survey weights: `ASECWT`
- Clustered standard errors: by `IND_2022`
- `post_unemployment = 1` for `YEAR >= 2023`
- `post_wage = 1` for `YEAR >= 2024`
- `early_career = 1` for ages 25-34
- `log_wage = log(INCWAGE)` for positive wage observations

For plots only, `AIIE` is also split into low- and high-exposure groups using the 25th and 75th percentiles. The regressions use continuous `AIIE`, not the grouped version.

In [22]:
```
from pathlib import Path
import random

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

plt.style.use("seaborn-v0_8-whitegrid")
pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

DATA_PATH = Path("../data/data_with_aiie.csv")
data = pd.read_csv(DATA_PATH)
```

In [23]:
```
df = data.copy()
```

```
df = df[df["AGE"].between(25, 64)].copy()
df = df[df["AIIE"].notna()].copy()

df["employed"] = df["EMPSTAT"].isin([10, 12]).astype(int)
df["unemployed"] = df["EMPSTAT"].isin([20, 21, 22]).astype(int)
df["in_labor_force"] = (df["LABFORCE"] == 2).astype(int)

df["post_unemployment"] = (df["YEAR"] >= 2023).astype(int)
df["post_wage"] = (df["YEAR"] >= 2024).astype(int)

df["early_career"] = df["AGE"].between(25, 34).astype(int)
df["older_worker"] = df["AGE"].between(50, 64).astype(int)

df["positive_wage"] = (df["INCWAGE"] > 0).astype(int)
df["log_wage"] = np.where(df["INCWAGE"] > 0, np.log(df["INCWAGE"]), np.nan)

q1 = df["AIIE"].quantile(0.25)
q3 = df["AIIE"].quantile(0.75)
df["exposure_group"] = np.select(
    [df["AIIE"] <= q1, df["AIIE"] >= q3],
    ["Low exposure", "High exposure"],
    default="Middle exposure",
)

unemp_sample = df[df["in_labor_force"] == 1].copy()
wage_sample = df[(df["employed"] == 1) & (df["positive_wage"] == 1)].copy()
wage_did_sample = wage_sample[wage_sample["YEAR"] != 2023].copy()
```

```
/Users/aeshagandhi/Downloads/MIDS-Sp26/UDS/UDS-
Project/.venv/lib/python3.14/site-packages/pandas/core/arraylike.py:402:
RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

In [24]:

```
def weighted_mean(series, weights):
    mask = series.notna() & weights.notna()
    if mask.sum() == 0:
        return np.nan
    return np.average(series[mask], weights=weights[mask])


def summarize_sample(frame, outcome_cols):
    rows = []
    for col in outcome_cols:
        rows.append(
            {
                "variable": col,
                "mean": frame[col].mean(),
                "std": frame[col].std(),
                "min": frame[col].min(),
                "max": frame[col].max(),
                "missing": frame[col].isna().sum(),
            }
        )
    return pd.DataFrame(rows)


def weighted_year_group_mean(frame, value_col, weight_col="ASECWT"):
    out = (
        frame.groupby(["YEAR", "exposure_group"], observed=True)
        .apply(lambda g: weighted_mean(g[value_col], g[weight_col]))
        .reset_index(name=value_col)
    )
    return out[out["exposure_group"].isin(["Low exposure", "High
exposure"])].copy()


def plot_trends(frame, value_col, title, ylabel):
    plot_df = weighted_year_group_mean(frame, value_col)
    fig, ax = plt.subplots(figsize=(9, 5))
    for group, group_df in plot_df.groupby("exposure_group"):
        ax.plot(group_df["YEAR"], group_df[value_col], marker="o",
label=group)
    ax.axvline(2023, color="black", linestyle="--", alpha=0.7, label="2023
```

```
unemployment post")
    ax.axvline(2024, color="gray", linestyle=":", alpha=0.9, label="2024
wage post")
    ax.set_title(title)
    ax.set_xlabel("Survey year")
    ax.set_ylabel(ylabel)
    ax.legend()
    plt.show()


def run_wls(formula, frame, cluster_col="IND_2022", weight_col="ASECWT"):
    model = smf.wls(formula=formula, data=frame, weights=frame[weight_col])
    return model.fit(cov_type="cluster", cov_kwds={"groups":
frame[cluster_col]})


def coef_table(result, keep=None):
    table = pd.DataFrame(
        {
            "coef": result.params,
            "std_err": result.bse,
            "p_value": result.pvalues,
        }
    )
    if keep is not None:
        mask = table.index.to_series().str.contains(keep, regex=True)
        table = table[mask]
    return table
```

Appendix C. Core Estimation Code

This section includes the code used for the descriptive plots, event-study diagnostics, main difference-in-differences regressions, and the age-heterogeneity specifications.

Difference-in-Differences Specifications

The baseline unemployment difference-in-differences model is:

$$unemployed_{it} = \beta_0 + \beta_1 AIIE_j + \beta_2 post_t^{unemp} + \beta_3 \left(AIIE_j \times post_t^{unemp}\right) + \beta_4 AGE_i \sum_e \delta_e 1\{EDUC_i = e\} + \varepsilon_{it}$$

The unemployment heterogeneity model adds a triple interaction:

$$unemployed_{it} = \alpha_0 + \alpha_1 AIIE_j + \alpha_2 post_t^{unemp} + \alpha_3 early_career_i + \alpha_4 \left(AIIE_j \times po + \alpha_5 \left(AIIE_j \times early_career_i\right) + \alpha_6 \left(post_t^{unemp} \times early_career_i\right) + \alpha_7 \left(AIIE_j \times post_t^{unemp} \times early_career_i\right) + \alpha_8 AGE_i + \sum_e \rho_e 1\{EDUC_i = e\} -$$

The baseline wage difference-in-differences model is:

$$\log(wage_{it}) = \gamma_0 + \gamma_1 AIIE_j + \gamma_2 post_t^{wage} + \gamma_3 \left(AIIE_j \times post_t^{wage}\right) + \gamma_4 AGE_i + \sum_e \kappa_e 1\{EDUC_i = e\} + \nu_{it}$$

The wage heterogeneity model parallels the unemployment triple-interaction setup:

$$\log(wage_{it}) = \eta_0 + \eta_1 AIIE_j + \eta_2 post_t^{wage} + \eta_3 early_career_i + \eta_4 \left(AIIE_j \times post_t^{wage} + \eta_5 \left(AIIE_j \times early_career_i\right) + \eta_6 \left(post_t^{wage} \times early_career_i\right) + \eta_7 \left(AIIE_j \times post_t^{wage} \times early_career_i\right) + \eta_8 AGE_i + \sum_e \phi_e 1\{EDUC_i = e\} + \omega_{it}$$

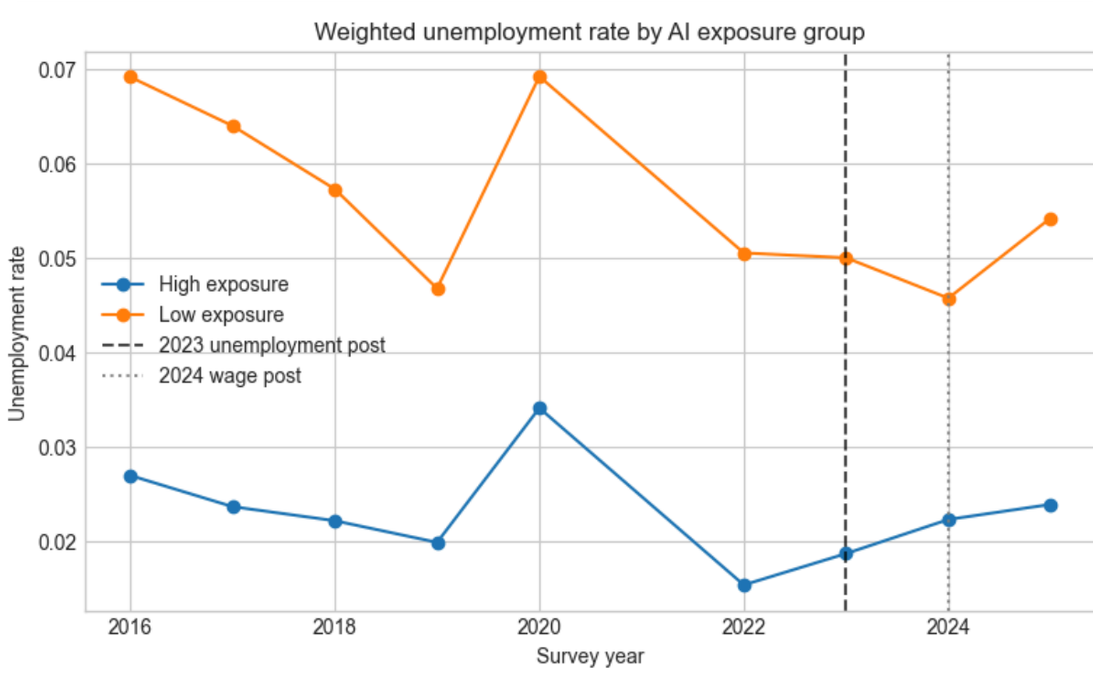
Variable definitions:

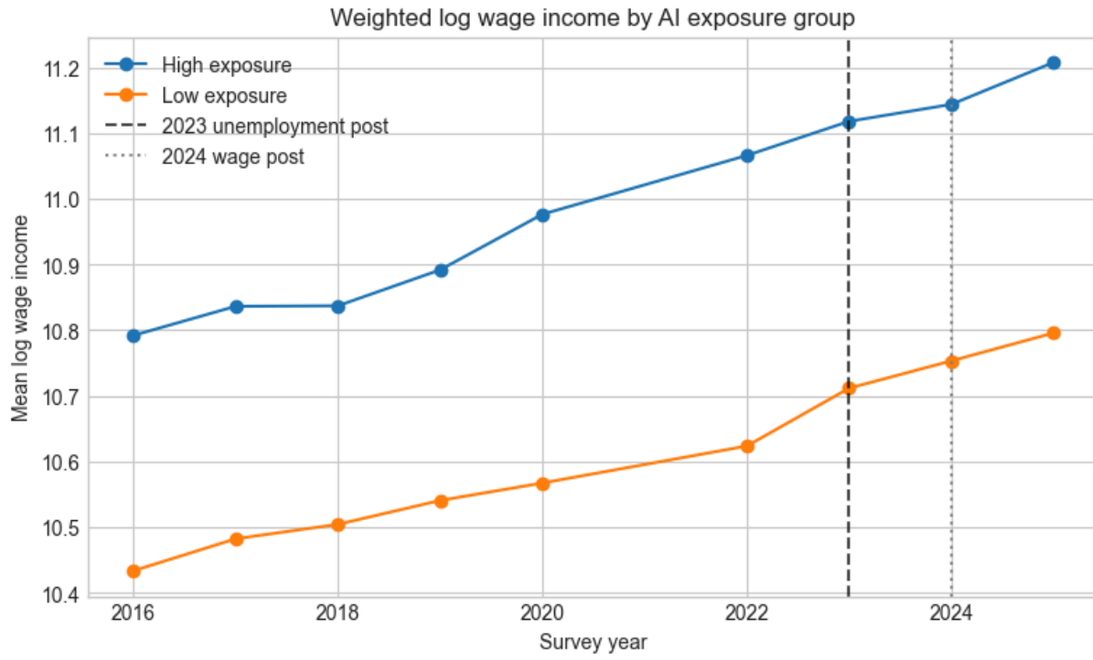
- $unemployed_{it}$: indicator equal to 1 if individual i is unemployed in year t
- $\log(wage_{it})$: log wage income for individual i in year t

- $AIIE_j$: AI Industry Exposure for industry j
- $post_t^{unemp}$: indicator equal to 1 in 2023-2025
- $post_t^{wage}$: indicator equal to 1 in 2024-2025
- $early_career_i$: indicator equal to 1 for ages 25-34
- AGE_i: age in years
- $1\{EDUC_i = e\}$: education-category controls

All models are estimated using CPS ASEC weights and cluster-robust standard errors at the 2022 Census industry level.

In [25]:
```
coverage_table = pd.DataFrame(
    {
        "sample": [
            "Working-age analysis frame",
            "Unemployment sample",
            "Wage sample",
            "Main wage DiD sample",
        ],
        "n_rows": [len(df), len(unemp_sample), len(wage_sample),
len(wage_did_sample)],
    }
)
coverage_table

plot_trends(
    unemp_sample,
    value_col="unemployed",
    title="Weighted unemployment rate by AI exposure group",
    ylabel="Unemployment rate",
)

plot_trends(
    wage_sample,
    value_col="log_wage",
    title="Weighted log wage income by AI exposure group",
    ylabel="Mean log wage income",
)
```





Parallel Trends and Event-Study Diagnostic

Parallel trends are evaluated with an event-study specification that replaces the single post-period interaction with year-specific interactions between `AIIE` and survey year:

$$Y_{it} = \theta_0 + \sum_{\tau \neq 2022} \theta_\tau \left(AIIE_j \times 1\{YEAR_t = \tau\} \right) + \sum_{\tau \neq 2022} \lambda_\tau 1\{YEAR_t = \tau\} + \sum_k \psi_k X$$

where Y_{it} is either unemployment or log wage income, and X_{kit} includes the controls used in the analysis. The omitted year is 2022, so each coefficient shows how the relationship between `AIIE` and the outcome differs relative to 2022.

Interpretation of the diagnostic:

- If pre-period coefficients are close to zero, that supports the parallel-trends assumption.
- If pre-period coefficients are systematically different from zero, that suggests pre-existing differential trends.
- The joint pre-trend test evaluates whether all pre-period `year x AIIE` terms are zero together.

In this project, the unemployment event study is more supportive of parallel trends than the wage event study, which is why the unemployment result is treated as the more credible causal estimate.

In [26]:
```
def fit_event_study(frame, outcome, reference_year=2022):
    formula = (
        f"{outcome} ~ C(YEAR, Treatment(reference={reference_year}))*AIIE +
"
        "C(IND_2022) + C(EDUC)"
    )
    return run_wls(formula, frame)


def extract_event_study_table(result, reference_year=2022):
    rows = []
    prefix = f"C(YEAR, Treatment(reference={reference_year}))"
    for name, coef in result.params.items():
        if not name.startswith(prefix) or ":AIIE" not in name:
            continue
        year = int(name.split("[T.")[1].split("]")[0])
        se = result.bse[name]
        rows.append(
            {
                "term": name,
```

```
                "year": year,
                "coef": coef,
                "std_err": se,
                "ci_low": coef - 1.96 * se,
                "ci_high": coef + 1.96 * se,
                "p_value": result.pvalues[name],
            }
        )
    return pd.DataFrame(rows).sort_values("year").reset_index(drop=True)


def joint_zero_test(result, terms):
    if len(terms) == 0:
        return pd.DataFrame({"n_terms": [0], "wald_stat": [np.nan],
"p_value": [np.nan]})
    param_names = list(result.params.index)
    r = np.zeros((len(terms), len(param_names)))
    for i, term in enumerate(terms):
        r[i, param_names.index(term)] = 1
    test = result.wald_test(r, scalar=True)
    return pd.DataFrame(
        {
            "n_terms": [len(terms)],
            "wald_stat": [float(test.statistic)],
            "p_value": [float(test.pvalue)],
        }
    )


def plot_event_study(table, title, post_start):
    fig, ax = plt.subplots(figsize=(9, 5))
    ax.errorbar(
        table["year"],
        table["coef"],
        yerr=1.96 * table["std_err"],
        fmt="o-",
        capsize=4,
    )
    ax.axhline(0, color="black", linewidth=1)
    ax.axvline(post_start, color="gray", linestyle="--", alpha=0.8)
    ax.set_title(title)
    ax.set_xlabel("Year")
    ax.set_ylabel("Coefficient on year x AIIE")
    plt.show()
```

In [ ]:

```
unemp_event = fit_event_study(unemp_sample, "unemployed",
reference_year=2022)
unemp_event_table = extract_event_study_table(unemp_event,
reference_year=2022)
display(unemp_event_table)

plot_event_study(
    unemp_event_table,
    title="Event-study coefficients for unemployment",
    post_start=2023,
)

unemp_pre_terms = unemp_event_table.loc[
    unemp_event_table["year"] < 2023, "term"
].tolist()
joint_zero_test(unemp_event, unemp_pre_terms)

wage_event = fit_event_study(wage_did_sample, "log_wage",
reference_year=2022)
wage_event_table = extract_event_study_table(wage_event,
reference_year=2022)
display(wage_event_table)

plot_event_study(
    wage_event_table,
    title="Event-study coefficients for log wage income",
    post_start=2024,
)
```

```
wage_pre_terms = wage_event_table.loc[
    wage_event_table["year"] < 2024, "term"
].tolist()
joint_zero_test(wage_event, wage_pre_terms)
```

In [ ]:

```
m_unemp_base = run_wls(
    "unemployed ~ AIIE * post_unemployment + AGE + C(EDUC)",
    unemp_sample,
)
coef_table(m_unemp_base, keep="AIIE|post")

m_unemp_triple = run_wls(
    "unemployed ~ AIIE * post_unemployment * early_career + AGE + C(EDUC)",
    unemp_sample,
)
coef_table(m_unemp_triple, keep="AIIE|post|early")

m_wage_base = run_wls(
    "log_wage ~ AIIE * post_wage + AGE + C(EDUC)",
    wage_did_sample,
)
coef_table(m_wage_base, keep="AIIE|post")

m_wage_triple = run_wls(
    "log_wage ~ AIIE * post_wage * early_career + AGE + C(EDUC)",
    wage_did_sample,
)
coef_table(m_wage_triple, keep="AIIE|post|early")
```

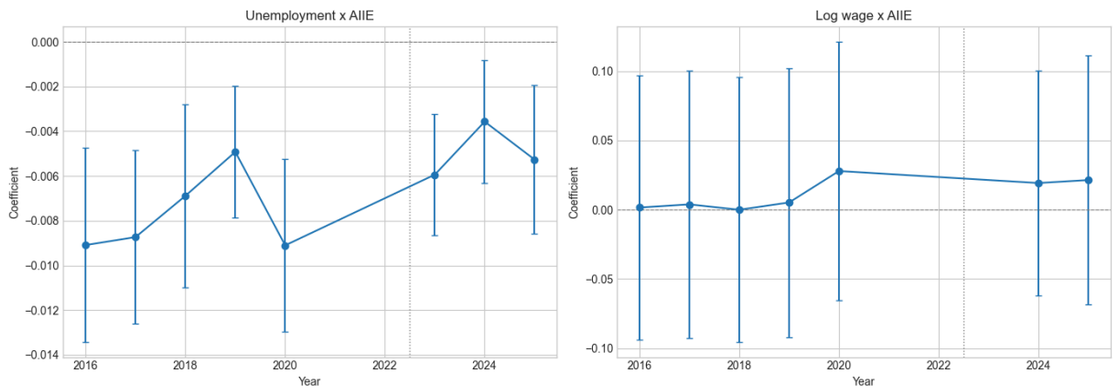
Out[ ]:

	coef	std_err	p_value
AIIE	0.0164	0.0497	0.7416
post_wage	0.2275	0.0088	0.0000
AIIE:post_wage	0.0025	0.0080	0.7542
early_career	-0.1816	0.0138	0.0000
AIIE:early_career	-0.0055	0.0205	0.7892
post_wage:early_career	0.0444	0.0093	0.0000
AIIE:post_wage:early_career	0.0089	0.0076	0.2403

In [ ]:

```
import re


def run_event_study(data, exposure, y_var, ax, title, ref_year=2022):
    data = data.copy()
    data["yr"] = pd.Categorical(data["YEAR"])
    m = run_wls(
        f"{y_var} ~ {exposure} : C(yr, Treatment(reference={ref_year}))"
        f" + C(yr) + AGE + C(EDUC)",
        data,
    )
    ec = m.params.filter(like=exposure)
    ci = m.conf_int().loc[ec.index]
    years = [int(re.search(r"\[(\d+)\]", s).group(1)) for s in ec.index]
    mask = [y != ref_year for y in years]
    years = [y for y, keep in zip(years, mask) if keep]
    ec, ci = ec[mask], ci[mask]

    ax.errorbar(
        years,
        ec.values,
        yerr=[(ec - ci.iloc[:, 0]).values, (ci.iloc[:, 1] - ec).values],
        fmt="o-",
        capsize=3,
    )
    ax.axhline(0, color="grey", ls="--", lw=0.8)
    ax.axvline(ref_year + 0.5, color="grey", ls=":", lw=1)
    ax.set_xlabel("Year")
```

```
    ax.set_title(title)


fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
run_event_study(unemp_sample, "AIIE", "unemployed", ax1, "Unemployment x
AIIE")
run_event_study(wage_did_sample, "AIIE", "log_wage", ax2, "Log wage x
AIIE")
ax1.set_ylabel("Coefficient")
ax2.set_ylabel("Coefficient")
plt.tight_layout()
plt.show()
```



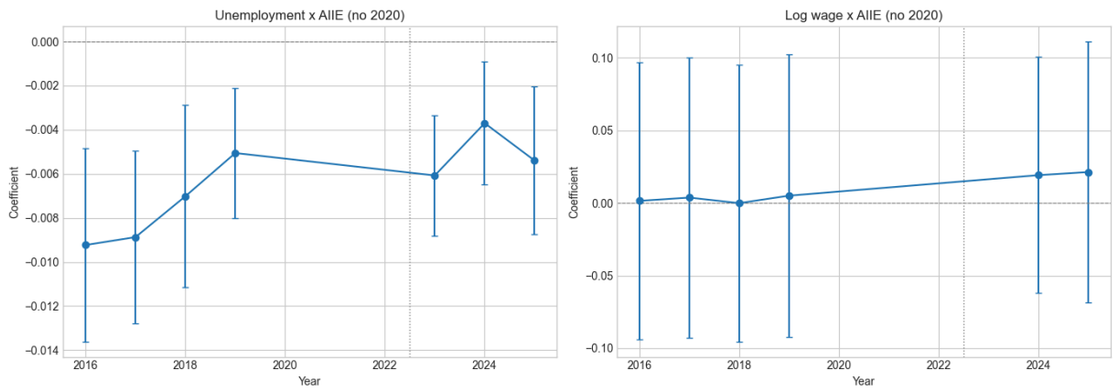
Appendix D. Robustness and Heterogeneity Code

The final blocks show the robustness exercise that excludes 2020 and the age-split specification that isolates whether the unemployment effect is driven more by workers ages 25-29 than by workers ages 30-34.

In [ ]:
```
unemp_no20 = unemp_sample[unemp_sample["YEAR"] != 2020].copy()
wage_no20 = wage_did_sample[wage_did_sample["YEAR"] != 2020].copy()

m_unemp_no20 = run_wls(
    "unemployed ~ AIIE * post_unemployment * early_career + AGE + C(EDUC)",
    unemp_no20,
)
m_wage_no20 = run_wls(
    "log_wage ~ AIIE * post_wage * early_career + AGE + C(EDUC)",
    wage_no20,
)

display(coef_table(m_unemp_no20, keep="AIIE|post|early"))
display(coef_table(m_wage_no20, keep="AIIE|post|early"))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
run_event_study(unemp_no20, "AIIE", "unemployed", ax1, "Unemployment x AIIE
(no 2020)")
run_event_study(wage_no20, "AIIE", "log_wage", ax2, "Log wage x AIIE (no
2020)")
ax1.set_ylabel("Coefficient")
ax2.set_ylabel("Coefficient")
plt.tight_layout()
plt.show()
```

	coef	std_err	p_value
AIIE	-0.0056	0.0019	0.0026
post_unemployment	-0.0027	0.0015	0.0649
AIIE:post_unemployment	0.0015	0.0013	0.2402
early_career	0.0089	0.0016	0.0000
AIIE:early_career	-0.0069	0.0016	0.0000
post_unemployment:early_career	-0.0020	0.0017	0.2416
AIIE:post_unemployment:early_career	0.0038	0.0014	0.0069

	coef	std_err	p_value
AIIE	0.0128	0.0499	0.7970
post_wage	0.2383	0.0090	0.0000
AIIE:post_wage	0.0061	0.0083	0.4616
early_career	-0.1787	0.0151	0.0000
AIIE:early_career	-0.0044	0.0208	0.8320
post_wage:early_career	0.0420	0.0097	0.0000
AIIE:post_wage:early_career	0.0078	0.0081	0.3353



In [ ]:
```
unemp_sample["age_group"] = pd.cut(
    unemp_sample["AGE"],
    bins=[24, 29, 34, 64],
    labels=["25-29", "30-34", "35-64"],
)

m_age_split = run_wls(
    "unemployed ~ AIIE * post_unemployment * C(age_group,
Treatment(reference='35-64'))"
    " + AGE + C(EDUC)",
    unemp_sample,
)

key_terms = [
    k
    for k in m_age_split.params.index
    if "AIIE" in k and "post" in k and "age_group" in k
]
coef_table(m_age_split).loc[key_terms]
```

Out[ ]:

	coef	std_err	p_value
AIIE:post_unemployment:C(age_group, Treatment(reference='35-64'))[T.25-29]	0.0047	0.0020	0.0198
AIIE:post_unemployment:C(age_group, Treatment(reference='35-64'))[T.30-34]	0.0022	0.0017	0.2016

Appendix E. Submission Export

The cell below uses `nbconvert` to export the notebook to HTML for submission. The memo body already contains markdown only, so no additional input-hiding step is needed there. Appendix code cells remain visible in the export.

In [ ]:
```
from pathlib import Path
import subprocess
import sys

notebook = Path('data_science_memo.ipynb')
output_name = 'data_science_memo_submission'

cmd = [
```

```python
    sys.executable,
    '-m',
    'jupyter',
    'nbconvert',
    '--to',
    'html',
    '--output',
    output_name,
    str(notebook),
]

print('Running:', ' '.join(cmd))
subprocess.run(cmd, check=True)
print(f'Created {output_name}.html')
```

```
Running: /Users/aeshagandhi/Downloads/MIDS-Sp26/UDS/UDS-
Project/.venv/bin/python -m jupyter nbconvert --to html --output
data_science_memo_submission data_science_memo.ipynb
```

```
[NbConvertApp] WARNING | Config option `exclude_output_stderr` not
recognized by `HTMLExporter`.  Did you mean one of: `exclude_output,
exclude_output_prompt, exclude_output_stdin`?
[NbConvertApp] WARNING | Config option `exclude_output_stderr` not
recognized by `HTMLExporter`.  Did you mean one of: `exclude_output,
exclude_output_prompt, exclude_output_stdin`?
[NbConvertApp] Converting notebook data_science_memo.ipynb to html
/Users/aeshagandhi/Downloads/MIDS-Sp26/UDS/UDS-
Project/.venv/lib/python3.14/site-packages/nbformat/__init__.py:96:
MissingIDFieldWarning: Cell is missing an id field, this will become a hard
error in future nbformat versions. You may want to use `normalize()` on your
notebooks before validations (available since nbformat 5.1.4). Previous
versions of nbformat are fixing this issue transparently, and will stop
doing so in the future.
  validate(nb)
[NbConvertApp] WARNING | Alternative text is missing on 12 image(s).
[NbConvertApp] Writing 773408 bytes to data_science_memo_submission.html
Created data_science_memo_submission.html
```

In [ ]: